Security Audit - Jupiter Lend DEX

conducted by Neodyme AG

Lead Auditor:	Benjamin Walny
Second Auditor & Administrative Lead:	Jasper Slusallek

June 25, 2026

Table of Contents

Appendices

1 | Executive Summary

Neodyme performed a security assessment of **Jupiter Lend DEX** program paired with an additional operational security (OpSec) review between May and June 2026 with two auditors and four person-weeks each. The DEX program lives within the Jupiter Lend Ecosystem and enables swapping between tokens while drawing liquidity from other programs in the ecosystem.

At the time of the audit, the DEX program and its related programs within the ecosystem were still under active development. Neodyme, however, was given a fixed commit, which was then reviewed. All auditing progressed smoothly without any interruptions or roadblocks.

The audit only uncovered one technical issue, which both only affects protocol revenue and would not lead to loss of funds; additionally, it was fixed while the audit was still underway. The general risk of the DEX program was found to be partially mitigated by its embedding into the Jupiter Lend architecture, via for instance having no direct custody over tokens and external oracle pricing. The general design was found to be rather lean, which helps greatly in security and auditability. The most critical parts of the DEX program remain swap pricing, its curve implementations and operational security.

The operational security stance, as discerned from both published multisig parameters and conversations with the development team, was reviewed. The review classifies the risk of compromise of the authorities and recommends a finer split of one of the main authorities of the program to increase risk tranching.

Overall, Neodyme concludes this test on a very positive note. Both from a technical and from an operational standpoint, Neodyme can attest awareness, knowledge and already-in-place security controls needed for secure operation.

2 | Introduction

From May 2026 until June 2026, Jupiter Lend engaged Neodyme to do a detailed security analysis of their Jupiter Lend DEX program. Two senior security researchers from Neodyme conducted independent full audits of the contract between the 4th of May 2026 and the 3rd of June 2026. Both auditors have a long track record of finding critical and other vulnerabilities in Solana programs, specializing in DeFi audits, having secured several billion dollars of TVL.

The main focus of this audit was the technical implementation security of the DEX smart contract. Here, the main focus was on key mathematical details such as the swap logic, the handling of respective shares and the special handling of debt based credit. This was paired with a review of the OpSec conducted via having an extensive interview of key personnel at Jupiter Lend to ensure a properly secured environment the contract can operate in. In the following sections, we present our findings, discuss worst-case scenarios for authority compromise, and provide some general notes for considerations that may be useful in the future.

The test progressed smoothly without interruptions and Jupiter Lend's team responded quickly and competently to questions of any kind via a shared communication channel. As perhaps evidenced by the number of findings, Jupiter Lend invested significant effort and resources into their product's security.

3 | Scope

The contract audit's scope comprised two major components:

- Primarily, the **implementation** security of the contract's source code
- Additionally, a review of the **operational security**

Neodyme considers the source code, located at https://github.com/Instadapp/fluid-contracts-solana, in scope for this audit. Third-party dependencies are not in scope. Jupiter Lend only relies on the Anchor library and the spl-token program, all of which are well-established. To ensure an environment for technical deep dives, the source code revision was fixed at the start of the audit.

The audited commit was **`fc533d70d6a3f1e1bbc7253d390e0dfc0e641597`**.

Additionally, he operational security of the authorities present within the DEX program was to be examined. Here, the goal is to uncover potential blind or weak spots, review current practices and advise on potential next steps to further harden the operation of the DEX contract. The scope includes here the on-chain configured authorities for the DEX program, their domain separation and the respective off-chain processes relating to the specific authorities. The OpSec of individuals has not been included in the scope. Further, other programs related to the Jupiter Lend ecosystem have not been part of the scope.

4 | Project Overview

This section briefly outlines Jupiter Lend DEX's functionality, outlines attack vectors and research questions Neodyme covered while auditing, followed by a detailed discussion of all related authorities and finally OpSec recommendations.

Functionality

The Jupiter DEX Program is an Anchor-based concentrated-liquidity AMM on Solana, a port of Instadapp's Fluid DEX on Ethereum. The DEX program specializes in swaps of token pairs with near-stable pricing. To reflect this, the program implements a Uniswap v3-like concentrated liquidity curve with real and imaginary reserves. The latter are used to concentrate liquidity within a price bound around a `center_price`, which is usually provided by an oracle.

The DEX custodies no tokens itself: it sits atop peer programs within the Jupiter Lend ecosystem: the **Liquidity layer** (the vault/bank that holds every token, accrues interest, and exposes `pre_operate`/`operate` CPIs), the **Oracle** program (which feeds `center_price` by aggregating sources) and a separate **Vaults** program.

Liquidity in one of the DEX's pools exists in two independent forms, both of which can be independently enabled by an admin: **smart collateral** (LP deposited as supply into the liquidity layer, earning yield) and **smart debt** (token0/token1 borrowed from the liquidity layer). Both forms of liquidity form independent sub-pools (and thus pricing curves) which can be traded against, and the contract tries to internally route swaps such that their pricing remains balanced. They can also be arbitraged against each other.

Besides the user facing swap IX, there are ten **pool management** instructions and twenty-four **admin configuration** instructions. Liquidity providers and integrating protocols manage positions through pool-management instructions: `deposit`/`withdraw` mint/burn collateral shares, and `borrow`/`payback` mint/burn debt shares. Each comes in a token-amount form (specify `token0_amt`/`token1_amt` plus a share-slippage bound, with imbalanced amounts internally swapped for a fee), a `*_perfect` form (specify exact shares, both tokens moving proportionally within token bounds), and single-token (`*_in_one_token`) variants. Administration uses a dual-authority model: a single `authority` controls authority/auths rotation, while an `auths` set (a set of at most 10 pubkeys) gates all operational instructions: pool creation (`init_dex`, `init_position`), enabling smart collateral/debt, and tuning fees, range/threshold percents, center-price source and limits, utilization caps, per-position supply/borrow configs, share caps, and a full set of pause controls (per-user, swap-and-arbitrage, and whole-DEX). A more detailed discussion of the authorities is delegated to the Operational Security section of this report.

Fees are two-tiered, both driven by **fee** and **revenue_cut**: the swap/LP fee discounts swap input before the curve so the withheld amount stays in reserves and accrues to LPs, while the revenue cut

is the protocol’s slice; on a swap the user transfers the full input into the vault but only the post-cut amount is credited to pool accounting, leaving the difference sitting uncredited in the Liquidity vault as protocol revenue.

5 | Methodology

Neodyme performed a full audit of the DEX program as defined in the Scope. In the following section we list some of the considered attack vectors.

General Solana Program Security: We checked the validation flaws most common to Solana and Anchor — missing ownership and signer checks, account confusions, weak account linkage, unsafe user-controlled bumps. Also reinitialization or account-creation DoS, missing rent-exemption assertions, non-unique seeds. No issues have been found in this regard, mostly attributed to correct and conservative anchor usage.

CPI Security: Given the reliance on the liquidity layer, both in- and outgoing CPI boundaries have been examined. The CPI targets have been reviewed; *liquidity program*: given mostly unchecked accounts on the DEX side, the receiving end has been checked whether, for instance `user_supply_position`/`user_borrow_position` match the respective dex, protocols and token reserve; *oracle program*: the `get_center_price` CPI and the `center_price_address` binding, source-account validation and the hop/exchange-rate aggregation, and whether oracle configuration enforces owned/verified source addresses rather than arbitrary accounts; *vault program*: as CPI into the DEX will originate from the vaults, the DEX side was checked for potential issues. Here, while not being a security issue, Neodyme recommends programmatically restricting the *DEX position* `protocols` to the vaults program as a further defense-in-depth step.

Mathematical arithmetics: All general math operations have been checked against generic pitfalls, checks include for instance rounding in favor of the protocol, dimensional analysis of all operands given the frequent usage of custom scales, integer over/underflow in unchecked cases where operands are not bound otherwise, review of `saturating_*` operations in contrast to the safe math library and for potential precision loss. Conservative rounding and continuous usage of both internally scaled integers and checked math lead to no exploitable issues in this regard.

AMM Pricing / Reserve Logic: One of the most critical parts of this audit iteration. Geometric-mean curve and its real/imaginary reserve derivation to mimic CLMM style trading for both collateral and debt pools, including the inversion path when the geometric mean exceeds price precision. Resulting curves have been re-constructed and verified for correctness, especially at the boundaries. The novelty here is that all available funds will be extended by imaginary funds, such that liquidity trades on a CLMM style linear curve around a center price. The three price shift mechanisms have been checked: admin config based range construction with upper/lower range, threshold-triggered shifting and center-price shifting: gradual time-based shifts, whether the center price is sourced from the external oracle or the stored value, and its clamping to the configured min/max bounds. No defects or angles to manipulate the pricing have been identified.

Swap Economics: Both the exact-in (`swap_in`) and exact-out (`swap_out`) paths were reviewed; beyond the standard slippage bounds, the maximum executable size is constrained by the out-token's real reserves and may never draw them below the minimum-liquidity floor. The per-swap oracle guard

received particular attention: it caps the stored-price move against the previous slot and makes all same-block swaps share a single budget while pinning the center price within a tight same-block band, so that neither a single flashloaned swap nor a multi-swap ratchet can walk the curve past its configured bounds. The routing split between the collateral and debt sub-pools was reconstructed and found exhaustive and symmetric - both legs carry identical fee and revenue cut, leaving no per-pool asymmetry to arbitrage, and the swap equalizes the two pools without a separate rebalancing step. A functional bug within the internal arbitrage has been shared but omitted from the report as it was not found to be security relevant. Fee handling was traced through to custody: the trading fee is withheld ahead of the curve in favor of the liquidity providers, while the revenue cut is retained as protocol revenue in the vault, with each intermediate rounding step biased toward the protocol. No exploitable value-extraction was identified. One logical flaw was found which only affected the collection of protocol revenue and is discussed in the findings section.

Authentication/Authorization: All IX have been checked to follow the intended AuthZ and proper AuthN via correct signer checks. Here, relying on very few shared anchor account validation structs helps not only in maintain- and audit-ability but also in security: no issues from missing or incorrect signer checks have been identified. A detailed discussion about the authorities in-use throughout the contract as well as hardening recommendations follow in the next section.

6 | Operational Security Review

As part of the audit, Neodyme reviewed Jupiter Lend's operational security as it pertains to the DEX contract.

The scope of this review was:

1. the categorization and risk classification of the different program authorities,
2. developing a potential suggestion for a more fine-grained authority split, and
3. a quick review of the team's current multisig and private key hygiene, their resilience against social engineering and their monitoring setup, as well as other key OpSec factors.

Number 1 was necessary to understand the risk that a compromise of each of the authorities represents. Number 2 was done to enable better separation of risk. Obviously, high-risk operations (e.g. operations that allow the operator to withdraw money) should be separated from operations that are done with less care (e.g. setting a non-vital parameter). Number 3 captured the general operational security stance.

We detail part of the results below. Due to operational security concerns, we do not publish all details. We begin with an overview of the contract's authorities:

Contract Authorities

The DEX program has the following relevant authorities:

- **Upgrade Authority:** As with any Solana contract, this authority can change the on-chain bytecode. If an attacker gets control of it, they can thus obviously steal all funds. Right now, the plan appears to be to point this to the global upgrade authority (`4MsgBB5VPoTrUSp5XnfbViV386C1UnsTdifLBw33ZMSJ`), which is a 4-of-7 multisig run by Jupiter and Fluid team members with a 12-hour time delay.
- **Program Init Authority:** According to documentation, this authority is meant to be temporary and is used to create new DEX trading pools or set up a DEX position for protocols. It is held by the Fluid team. While it is copied across other lend protocols and hasn't been removed yet, its power is very limited. If compromised, an attacker could frontrun pool creation or lock up the authority account made by `init_position`, causing a limited DoS. They cannot steal funds or even turn on swapping. However, note that it can also create a position which can then be activated by one of the `dex_admin.auths` authorities.
- **`dex_admin.authority`:** This can only replace itself using `update_authority` and it can change the keys in the admin list using `update_auths`. Its actual power matches whatever the keys in the admin list can do (since it can add itself), except it also has the power to kick keys out of that list.
- **`dex_admin.auths` List:** This is a list of up to 10 keys. Every key on this list can run all gated admin commands described in the next section. Right now, if any single key on this list is compromised,

it can lead to a total loss of funds. The team stated that they ideally plan to split this list into smaller, more specific roles based on the worst-case effect of a key compromise.

The main improvement that is possible is a better differentiation of the risk of the admin instructions controlled by the pubkeys in `dex_admin.auths`. We propose one in the next section.

Admin Instructions Risk Classification

After review, we would recommend the following split of instructions into high-risk and lower-risk instructions which can be executed by the pubkeys in `dex_admin.auths`. High-risk instructions should be handled by a well-protected authority, ideally a multisig. If for whatever reason one of the high-risk instructions has to be executed often or even automatically, great care should be taken to limit the effects of a compromise by other means.

The Fluid Lend team also mentioned that they may want to gate the changing of parameters behind a separate contract that enforces certain rules and possibly timelocks for parameter changes.

Tier 1: High Risk (Can cause loss of funds or needs strict protection)

- `update_center_price_address`: Sets the external price feed (oracle), along with how quickly the price is shifted. If this were compromised, the attacker could simply change the center price, which is obviously critical.
- `update_range_percents`: This shifts the liquidity concentration around the center price, which allows an attacker to warp swap pricing.
- `update_center_price_limits`: Forces prices to cap out if they move outside a specific window, warping how trades are priced.
- `turn_on_smart_debt`: Directly lets the attacker withdraw funds, causing an obvious loss of money.
- `turn_on_smart_col`: Turns on smart collateral trading for a new pool. This doesn't directly steal money, but this is a major system change that should still be put into Tier 1.
- `Unpausing`: While unpausing the protocol (or any of the other more fine-grained pausing knobs) doesn't directly steal money, a paused contract may mean an exploit is actively happening. Restarting the system should be restricted to a highly secure key.
- `update_threshold_percent`: While there's no direct loss-of-funds if compromised, this is part of the pricing logic and should likely still be highly secure.
- `init_position`: This creates an isolated position for a protocol; it still needs to be activated (e.g. for borrowing) to be able to do much. However, note that this can be done using `update_user_borrow_config` and the related instructions, which is why we recommend gating this instruction within Tier 1.

Tier 2: Lower Risk

- `update_fee_and_revenue_cut`: Can be used to make users pay higher fees or stop trades, but an attacker cannot directly steal funds with this instruction.
- Pausing (these are separate instructions for the DEX, users, and swaps and arbitrage): This stops the protocol from working, but the command needs to be easy to use so the team can freeze the system quickly during an exploit.
- `update_utilization_limit`, `update_user_supply_config`, `update_user_withdrawal_limit`: these are simply parameters for lending from the liquidity layer and for the protocol usage, they do not appear critical.
- `update_user_borrow_config`: This sets a protocol's borrow config, much like the parameter updates from the last bullet point. However, note that it should be considered whether it's worthwhile to gate `max_debt_ceiling` separately.
- `init_dex`: This cannot do much until the pool is actually filled with money.
- `update_max_supply_shares` and `update_max_borrow_shares`: Both of these are simply caps on the share count, at worst they can cause a DoS if set too low.

Jupiter keeps a public list of its multisigs and authorities, which also apply to the other Jupiter Lend systems, at https://jup.ag/lend/transparency#multisigs

Protecting Against Authority Compromises

Neodyme also discussed Jupiter Lend's general OpSec stance with several members of their team as it pertains to the DEX contract's authorities.

Of primary importance in protecting against authority compromises is private key hygiene. We recommended an entirely separate device that is used only for signing, with an additional hardware wallet layer to make key extraction harder. We also recommended having sufficiently large subsets of the multisig that use different vendors and frameworks for protecting their keys in order to eliminate single points of failure.

Jupiter stated that they currently have a multi-vendor setup, with most signatories using hardware wallets.

Of course, transactions that are signed must also be verified by each multisig participant. This step is often skipped in multisigs due to capacity constraints and the effort involved, but it's essential. Many past high-profile authority compromises could have been avoided by proper checking of the transactions that were signed.

Jupiter stated that they have a separate internal UI for Squads for non-technical people, as well as custom code for displaying and verifying transaction data, which is not shared between signers. They also stated that for upgrade transactions, the code will be verified both internally and by external auditors.

Monitoring is essential as well, and Jupiter stated that they have it set up for oracle liveness and discrepancies from trusted sources, for whale activity and for multisig actions that are taken. Individual transaction monitoring is not currently set up, but they stated that they are exploring this.

Neodyme also had a long and fruitful discussion with members of the Jupiter team about choice and security of their multisig framework, trusting RPC, key backups, succession planning, break-glass plans and security of communication channels, among other things. The details are omitted in this report, but the auditors would like to emphasize that in almost all domains, the Jupiter team was knowledgeable and aware of best practices.

7 | Findings

This section outlines all of our findings. They are classified into one of five severity levels, detailed in Appendix C.

In addition to this finding, Neodyme delivered the Jupiter Lend team non direct security related found issues and general additional notes which are not part of this report.

All findings are listed in Table 1 and further described in the following sections.

Identifier	Name	Severity	Status
ND-JPL-LO-01	Swap out collects zero protocol revenue	LOW	Fixed

Table 1: Findings

[ND-JPL-LO-01] Swap out collects zero protocol revenue

Severity	Impact	Affected Component	Status
LOW	Loss of protocol revenue	DEX	Fixed

Description

Auditing the fee handling throughout the dex codebase, the swap_out IX was found to incorrectly handles the fees, leading to missing revenue for the protocol. When calculating the swap amounts and respective token split between the collateral and debt pools, the fee are added on top and end up as untracked token assets in the Liquidity Layer. In the swap_out IX however, it was found that the resulting amount to be deducted from the user did only account for the respective collateral and debt shares but did not correctly incorporate the revenue cut.

Location

https://github.com/Instadapp/fluid-contracts-solana/blob/fc533d70d6a3f1e1bbc7253d390e0dfc0e641597/programs/dex/src/utils/swap.rs#L430

Relevant Code

```
pub fn execute_swap_out(
[...]
    // Exact-out: round user input up so native transfers cover 9-dec ceil fee math.
    let col_deposit_native = col_deposit.descale_up(in_decimals)?;
    let debt_payback_native = debt_payback.descale_up(in_decimals)?;
    let col_withdraw_native = amt_out_col.descale(out_decimals)?;
    let debt_borrow_native = amt_out_debt.descale(out_decimals)?;

    let total_in = col_deposit_native.safe_add(debt_payback_native)?;

    if total_in > amount_in_max.cast()? {
        return Err(error!(ErrorCodes::DexExceedsAmountInMax));
    }
```

Mitigation Suggestion

Instead of using the LL tracked totals, add the calculated input amounts (`amt_in_col_native`, `amt_in_debt_native`) for the new `total_in` and revise the final call to the LL via `deposit_or_payback_liquidity` to include the calculated `transfer_amount`.

Remediation

The issues has been remediated during the audit period and the fix has been verified via commit: 69f7e11da549082ec20b29371ce20f3b325693d9

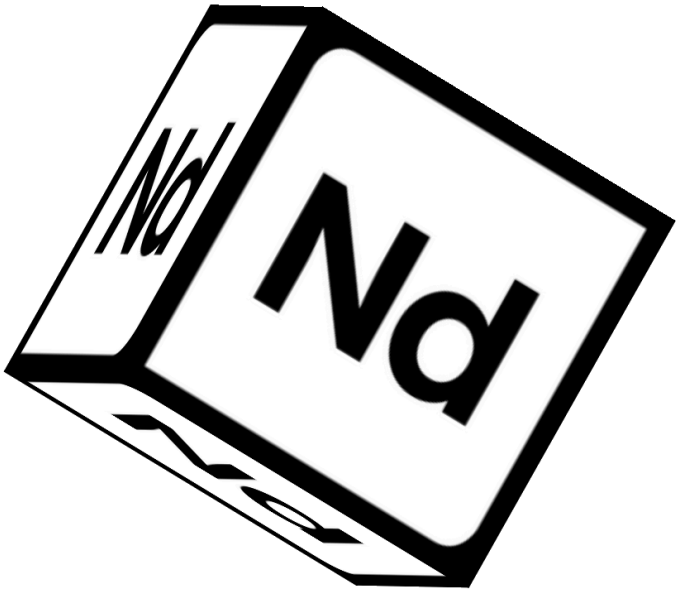
A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over $10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events worldwide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.



B | Methodology

We are not checklist auditors.

In fact, we pride ourselves on that. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated.

We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to find bugs that others miss. We often extend our audit to cover off-chain components in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list here.

Select Common Vulnerabilities

Our most common findings are still specific to Solana itself. Among these are vulnerabilities such as the ones listed below:

- Insufficient validation, such as:
  - Missing ownership checks
  - Missing signer checks
  - Signed invocation of unverified programs
  - Account confusions
  - Missing freeze authority checks
  - Insufficient SPL account verification
  - Dangerous user-controlled bumps
  - Insufficient Anchor account linkage
- Account reinitialization vulnerabilities
- Account creation DoS
- Redeployment with cross-instance confusion
- Missing rent exemption assertion
- Casting truncation
- Arithmetic over- or underflows
- Numerical precision and rounding errors
- Anchor pitfalls, such as accounts not being reloaded
- Non-unique seeds
- Issues arising from CPI recursion
- Log truncation vulnerabilities
- Vulnerabilities specific to integration of Token Extensions, for example unexpected external token hook calls

Apart from such Solana-specific findings, some of the most common vulnerabilities relate to the general logical structure of the contract. Specifically, such findings may be:

- Errors in business logic
- Mismatches between contract logic and project specifications
- General denial-of-service attacks
- Sybil attacks
- Incorrect usage of on-chain randomness
- Contract-specific low-level vulnerabilities, such as incorrect account memory management
- Vulnerability to economic attacks
- Allowing front-running or sandwiching attacks

Miscellaneous other findings are also routinely checked for, among them:

- Unsafe design decisions that might lead to vulnerabilities being introduced in the future
    - Additionally, any findings related to code consistency and cleanliness
- Rug pull mechanisms or hidden backdoors

Often, we also examine the authority structure of a contract, investigating their security as well as the impact on contract operations should they be compromised.

Over the years, we have found hundreds of high and critical severity findings, many of which are highly nontrivial and do not fall into the strict categories above. This is why our approach has always gone way beyond simply checking for common vulnerabilities. We believe that the only way to truly secure a program is a deep and tailored exploration that covers all aspects of a program, from small low-level bugs to complex logical vulnerabilities.

C | Vulnerability Severity Rating

We use the following guideline to classify the severity of vulnerabilities. Note that we assess each vulnerability on an individual basis and may deviate from these guidelines in cases where it is well-founded. In such cases, we always provide an explanation.

Severity	Description
CRITICAL	Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.
HIGH	Bugs that can be used to set up loss of funds in a more limited capacity, or to render the contract unusable.
MEDIUM	Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms, or that could be exploited to render the contract partially unusable.
LOW	Bugs that do not have a significant immediate impact and could be fixed easily after detection.
INFORMATIONAL	Bugs or inconsistencies that have little to no security impact, but are still noteworthy.

Additionally, we often provide the client with a list of nit-picks, i.e. findings whose severity lies below Informational. In general, these findings are not part of the report.

Neodyme AG

Dirnismaning 55
Halle 13
85748 Garching
Germany

E-Mail: contact@neodyme.io

https://neodyme.io